

TensorFlow.js：將 Python SavedModel 轉換為 TensorFlow.js 格式

來源：<https://codelabs.developers.google.com/codelabs/tensorflowjs-convert-python-savedmodel?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將學習如何使用 mod 格式的現有 Python 機器學習模型並將其轉換為 TensorFlow.js 格式，以便在網路瀏覽器中執行，同時瞭解如何解決轉換期間的常見問題。

目錄

1. [簡介](#)
2. [什麼是 TensorFlow.js ?](#)
3. [設定系統](#)
4. [製作 Python 模型](#)
5. [將 SavedModel 轉換為 TensorFlow.js 格式](#)
6. [在瀏覽器中使用轉換後的模型](#)
7. [無法轉換的模型](#)
8. [恭喜](#)

1. 簡介

您已使用 [TensorFlow.js](#) 踏出第一步，試用[預先建構的模型](#)，甚至可能已自行建構模型，但您發現 Python 推出一些尖端研究，因此想知道這些研究是否能在網路瀏覽器中執行，讓您實現酷炫的想法，並以可擴充的方式提供給數百萬人使用。聽起來是不是很熟悉？如果是，這個程式碼研究室就是為您而設！

TensorFlow.js 團隊開發了便利的工具，可透過指令列轉換工具，將 [SavedModel](#) 格式的模型轉換為 TensorFlow.js，讓您在網路上使用這些模型時，享有觸及範圍和規模優勢。

課程內容

在本程式碼研究室中，您將瞭解如何使用 TensorFlow.js 指令列轉換工具，將 Python 產生的 SavedModel 移植到 model.json 格式，以便在網路瀏覽器中於用戶端執行。

詳細說明：

- 如何建立簡單的 Python 機器學習模型，並儲存為 TensorFlow.js 轉換器所需的格式。
- 如何安裝及使用 TensorFlow.js 轉換工具，將從 Python 匯出的 SavedModel 轉換為 TensorFlow.js 模型。
- 從轉換作業取得結果檔案，並在 JS 網頁應用程式中使用。
- 瞭解模型無法轉換時該怎麼做，以及可用的選項。

試想一下，您能將新發布的研究成果提供給全球數百萬名 JS 開發人員使用。或者，您也可以在自己的創作中使用這項技術，只要在網路瀏覽器中執行，世界各地的使用者都能體驗，因為不需要複雜的依附元件或環境設定。準備好開始駭客松了嗎？立即開始！

請注意：本程式碼研究室的重點在於如何使用及開始使用 TensorFlow.js 轉換器，因此模型建立本身會使用樣板程式碼，以便取得可在教學課程中使用的檔案。如要進一步瞭解如何製作自己的模型，請參閱[其他程式碼研究室](#)。

歡迎與我們分享轉換後的檔案！

您可以運用今天所學，嘗試從 Python 轉換一些喜愛的模型。如果順利完成，並建立模型運作的示範網站，請在社群媒體上使用 **#MadeWithTFJS** 主題標記標記我們，您的專案就有機會登上 [TensorFlow 網誌](#)，甚至是日後的[展演活動](#)。我們很期待看到更多出色的研究成果移植到網路上，讓更多人能以創新或創意的方式使用這類模型，就像[這個絕佳範例](#)一樣。

2. 什麼是 TensorFlow.js ?



[TensorFlow.js](#) 是開放原始碼機器學習程式庫，可在 JavaScript 執行的任何位置執行。這個程式庫是以原始的 [Python 版 TensorFlow](#) 程式庫為基礎，目標是為 JavaScript 生態系統重新建立這種開發人員體驗和 API 集。

哪些類別可以套用顯示設定？

由於 JavaScript 具有可攜性，您現在只要使用 1 種語言，就能輕鬆在下列所有平台執行機器學習：

- 網頁瀏覽器中的用戶端，使用原生 JavaScript
- 伺服器端，甚至是 **Raspberry Pi** 等物聯網裝置 (使用 Node.js)
- 使用 Electron 的電腦應用程式
- 使用 React Native 的原生行動應用程式

TensorFlow.js 也支援這些環境中的多個後端 (例如 CPU 或 WebGL 等實際硬體環境)。在此情況下，「後端」並非指伺服器端環境，而是指執行後端 (例如 WebGL 中的用戶端)，確保相容性並維持快速運作。TensorFlow.js 目前支援：

- **在裝置的顯示卡 (GPU) 上執行 WebGL**：這是執行較大型模型 (超過 3 MB) 的最快方式，可透過 GPU 加速。
- **在 CPU 上執行 Web Assembly (WASM)** - 提升各種裝置的 CPU 效能，包括舊款手機。這類模型較小 (小於 3 MB)，由於將內容上傳至圖形處理器的負擔，這類模型在 CPU 上使用 WASM 執行的速度，實際上會比使用 WebGL 更快。
- **CPU 執行** - 如果沒有其他環境可用，則應採用這個備援方案。這是三者中最慢的，但隨時可供使用。

注意：如果您知道要執行的裝置，可以選擇[強制使用其中一個後端](#)，也可以不指定，讓 TensorFlow.js 為您決定。

用戶端超能力

在用戶端電腦的網路瀏覽器中執行 TensorFlow.js，可帶來多項值得考慮的優點。

隱私權

您可以在用戶端電腦上訓練及分類資料，完全不必將資料傳送至第三方網路伺服器。有時，這可能是為了遵守當地法律 (例如 GDPR) 的規定，或是處理使用者想保留在電腦上，不想傳送給第三方的資料。

速度

由於不必將資料傳送至遠端伺服器，推論 (分類資料的行為) 速度會更快。更棒的是，如果使用者授予存取權，您就能直接存取裝置的感應器，例如相機、麥克風、GPS、加速度計等。

觸及率和規模

只要按一下連結，世界各地的使用者就能在瀏覽器中開啟網頁，並使用您製作的內容。您不必為了使用機器學習系統，在伺服器端進行複雜的 Linux 設定，包括 CUDA 驅動程式等。

費用

由於沒有伺服器，您只需支付 CDN 費用，即可代管 HTML、CSS、JS 和模型檔案。與讓伺服器 (可能附有顯示卡) 全天候運作相比，CDN 的成本便宜許多。

伺服器端功能

運用 Node.js 實作的 TensorFlow.js 可啟用下列功能。

完整支援 CUDA

在伺服器端，如要使用顯示卡加速，必須安裝 [NVIDIA CUDA 驅動程式](#)，才能讓 TensorFlow 與顯示卡搭配運作 (與使用 WebGL 的瀏覽器不同，不需要安裝)。不過，如果完全支援 CUDA，就能充分運用顯示卡的低階功能，進而縮短訓練和推論時間。由於兩者共用相同的 C++ 後端，因此效能與 Python TensorFlow 實作項目相同。

模型大小

如果是研究領域的尖端模型，您可能會使用非常大的模型，大小可能達到 GB 級。由於每個瀏覽器分頁的記憶體用量有限制，目前無法在網路瀏覽器中執行這些模型。如要執行這些較大的模型，您可以在自己的伺服器上使用 Node.js，並具備有效執行這類模型所需的硬體規格。

IOT

Node.js 支援 Raspberry Pi 等熱門單板電腦，因此您也可以在这些裝置上執行 TensorFlow.js 模型。

速度

Node.js 是以 JavaScript 編寫而成，因此可從即時編譯中獲益。這表示使用 Node.js 時，您通常會發現效能有所提升，因為系統會在執行階段進行最佳化，特別是針對您可能執行的任何前置處理。如需這方面的絕佳範例，請參閱[這份個案研究](#)，瞭解 Hugging Face 如何使用 Node.js，將自然語言處理模型的效能提升一倍。

現在您已瞭解 TensorFlow.js 的基本概念、執行位置和一些優點，接下來就開始使用這項技術完成實用工作吧！

步驟 3 · 約 10 分鐘

3. 設定系統

在本教學課程中，我們將使用 Ubuntu。這是許多人使用的熱門 Linux 發行版本，如果您選擇在雲端虛擬機器上操作，Google Cloud [Compute Engine](#) 也提供 Ubuntu 做為基礎映像檔。

撰寫本文時，我們可以在[建立新的原始 Compute Engine 執行個體](#)時選取 Ubuntu 18.04.4 LTS 映像檔，這也是我們將使用的映像檔。您當然可以使用自己的電腦，甚至選擇使用其他作業系統，但不同系統的安裝說明和依附元件可能有所不同。

安裝 TensorFlow (Python 版本)

現在，您可能正嘗試轉換找到或將編寫的現有 Python 型模型，但如要從 Python 匯出「SavedModel」檔案，您必須先在執行個體上設定 Python 版 TensorFlow (如果「SavedModel」尚未開放下載)。

[透過 SSH 連線至您在上方建立的雲端機器](#)，然後在終端機視窗中輸入下列內容：

終端機視窗：

```
sudo apt update
sudo apt-get install python3
```

這可確保機器上已安裝 Python 3。如要使用 TensorFlow，必須安裝 Python 3.4 以上版本。

如要確認安裝的版本是否正確，請輸入下列內容：

終端機視窗：

```
python3 --version
```

您應該會看到一些輸出內容，指出版本號碼，例如 `Python 3.6.9`。如果列印結果正確，且值大於 3.4，即可繼續。

接下來，我們要為 Python 3 安裝 PIP (Python 的套件管理員)，然後更新 PIP。Type:

終端機視窗：

```
sudo apt install python3-pip
pip3 install --upgrade pip
```

我們同樣可以透過以下指令驗證 pip3 安裝作業：

終端機視窗：

```
pip3 --version
```

撰寫本文時，我們發現執行這項指令後，終端機上會顯示 `pip 20.2.3`。

安裝 TensorFlow 前，請確認 Python 套件「setuptools」為 41.0.0 以上版本。執行下列指令，確保已更新至最新版本：

終端機視窗：

```
pip3 install -U setuptools
```

最後，我們現在可以安裝 Python 適用的 TensorFlow：

終端機視窗：

```
pip3 install tensorflow
```

這項作業需要一段時間才能完成，請等待執行完畢。

警告：在本教學課程中，我們將安裝僅限 CPU 的 TensorFlow 版本，以便快速設定。如要使用執行個體的 GPU，您需要安裝 TensorFlow 的 GPU 版本，這樣訓練時間會快得多。不過，這個版本需要安裝 NVIDIA CUDA 驅動程式，這超出本程式碼研究室的範圍。如要進一步瞭解如何安裝 GPU 版 TensorFlow，請參閱[這篇文章](#)。

請檢查 TensorFlow 是否已正確安裝。在目前目錄中建立名為 `test.py` 的 Python 檔案：

終端機視窗：

```
nano test.py
```

開啟 nano 後，我們可以編寫一些 Python 程式碼，列印已安裝的 TensorFlow 版本：

test.py:

```
import tensorflow as tf
print(tf.__version__)
```

按下 `CTRL + O` 將變更寫入磁碟，然後按下 `CTRL + X` 退出 nano 編輯器。

現在我們可以執行這個 Python 檔案，查看螢幕上顯示的 TensorFlow 版本：

終端機視窗：

```
python3 test.py
```

撰寫本文時，我們看到控制台列印了 `2.3.1`，表示已安裝 TensorFlow Python 版本。

警告：如果看到許多與 NumPy 相關的警告彈出視窗，請在終端機中輸入下列內容，升級 NumPy 版本：

```
pip uninstall numpy
```

```
pip install numpy==1.16.4
```

您可能仍會看到一些有關 GPU / libcudart 的警告訊息，這是正常的，因為我們安裝的是 TensorFlow 的 CPU 版本，而非 GPU 版本。您可以忽略這些訊息。

4. 製作 Python 模型

本程式碼研究室的下一個步驟將逐步說明如何建立簡單的 Python 模型，並示範如何以「SavedModel」格式儲存訓練完成的模型，然後搭配 TensorFlow.js 指令列轉換器使用。如要轉換任何 Python 模型，原理都類似，但我們會盡量簡化這段程式碼，方便大家瞭解。

請編輯在第一節中建立的 test.py 檔案，並將程式碼更新如下：

test.py:

```
import tensorflow as tf
print(tf.__version__)

# Import NumPy - package for working with arrays in Python.
import numpy as np

# Import useful keras functions - this is similar to the
# TensorFlow.js Layers API functionality.
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

# Create a new dense layer with 1 unit, and input shape of [1].
layer0 = Dense(units=1, input_shape=[1])
model = Sequential([layer0])

# Compile the model using stochastic gradient descent as optimiser
# and the mean squared error loss function.
model.compile(optimizer='sgd', loss='mean_absolute_error')

# Provide some training data! Here we are using some fictional data
# for house square footage and house price (which is simply 1000x the
# square footage) which our model must learn for itself.
xs = np.array([800.0, 850.0, 900.0, 950.0, 980.0, 1000.0, 1050.0, 1075.0, 1100.0, 1150.0,
1200.0, 1250.0, 1300.0, 1400.0, 1500.0, 1600.0, 1700.0, 1800.0, 1900.0, 2000.0], dtype=float)

ys = np.array([800000.0, 850000.0, 900000.0, 950000.0, 980000.0, 1000000.0, 1050000.0,
1075000.0, 1100000.0, 1150000.0, 1200000.0, 1250000.0, 1300000.0, 1400000.0, 1500000.0,
1600000.0, 1700000.0, 1800000.0, 1900000.0, 2000000.0], dtype=float)

# Train the model for 500 epochs.
model.fit(xs, ys, epochs=500, verbose=0)

# Test the trained model on a test input value
print(model.predict([1200.0]))

# Save the model we just trained to the "SavedModel" format to the
# same directory our test.py file is located.
tf.saved_model.save(model, './')
```

這段程式碼會訓練非常簡單的線性迴歸，學習估算所提供 x (輸入) 和 y (輸出) 之間的關係。接著，我們會將訓練好的模型儲存到磁碟。如要進一步瞭解每一行的作用，請查看內嵌註解。

執行這個程式後，如果我們檢查目錄 (呼叫 `python3 test.py`)，現在應該會在目前的目錄中看到一些新檔案和資料夾：

- `test.py`
- `saved_model.pb`
- 資產
- `variables`

我們現在已產生 TensorFlow.js 轉換工具所需的檔案，可將這個模型轉換為在瀏覽器中執行！

步驟 5 · 約 10 分鐘

5. 將 SavedModel 轉換為 TensorFlow.js 格式

安裝 TensorFlow.js 轉換工具

如要安裝轉換器，請執行下列指令：

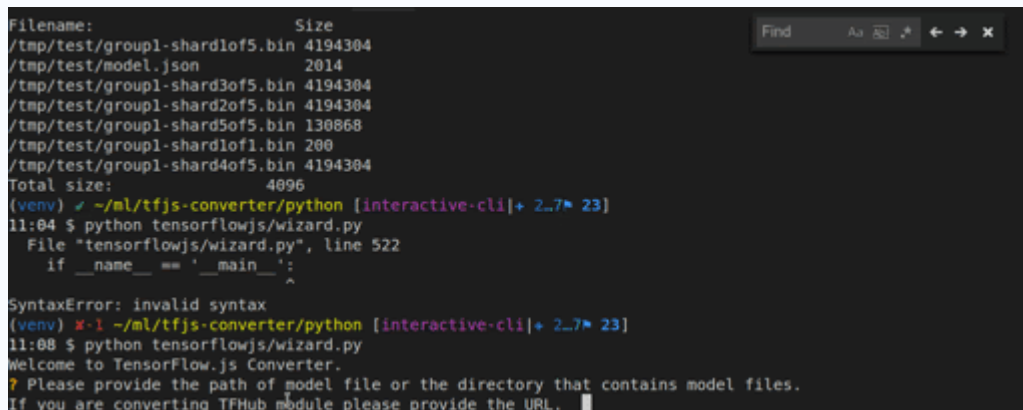
終端機視窗：

```
pip3 install tensorflowjs
```

這很簡單。

請注意：您也可以使用 `pip3 install tensorflowjs[wizard]` 安裝精靈

安裝完成後，您就可以執行 `tensorflowjs_wizard` 指令，以更友善的方式完成轉換程序，如下所示：



The screenshot shows a terminal window with a file listing and command execution. The file listing shows several files in a directory, including model shards and a model.json file. The command execution shows the user running `python tensorflowjs/wizard.py`, which results in a `SyntaxError: invalid syntax`. The user then runs `python tensorflowjs/wizard.py` again, which results in a `Welcome to TensorFlow.js Converter.` message and a prompt for the path of the model file or directory.

假設我們使用的是指令列轉換器 (`tensorflowjs_converter`)，而非上述精靈版本，我們可以呼叫下列指令來轉換剛建立的已儲存模型，並明確將參數傳遞至轉換器：

終端機視窗：

```
tensorflowjs_converter \
  --input_format=keras_saved_model \
  ./ \
  ./predict_houses_tfjs
```

這是怎麼回事？首先，我們要呼叫剛才安裝的 `tensorflowjs_converter` 二進位檔，並指定要轉換 Keras 已儲存的模型。

在上述範例程式碼中，您會發現我們匯入了 Keras，並使用其高階層 API 建立模型。如果 Python 程式碼未使用 Keras，建議使用其他輸入格式：

- **keras** - 載入 Keras 格式 (HDF5 檔案類型)
- **tf_saved_model**：載入使用 TensorFlow Core API (而非 Keras) 的模型。
- **tf_frozen_model** - 載入含有凍結權重的模型。
- **tf_hub** - 載入從 TensorFlow Hub 產生的模型。

如要進一步瞭解其他格式，請參閱[這篇文章](#)。

接下來的 2 個參數會指定儲存模型的資料夾位置。在上述範例中，我們指定了目前的目錄，最後指定要將轉換作業輸出至哪個目錄，也就是目前目錄中名為「`predict_houses_tfjs`」的資料夾。

執行上述指令後，目前目錄中會建立名為 `predict_houses_tfjs` 的新資料夾，內含：

- `model.json`
- `Group1-shard1of1.bin`

這些是我們在網頁瀏覽器中執行模型時需要的檔案。請儲存這些檔案，我們會在下一節中使用。

請注意：用戶端 TensorFlow.js 檔案一律會包含 `model.json` 和至少一個 `.bin` 檔案。二進位檔會分成 4 MB 大小的區塊，以便將這類素材資源平行傳送至用戶端的網頁瀏覽器。

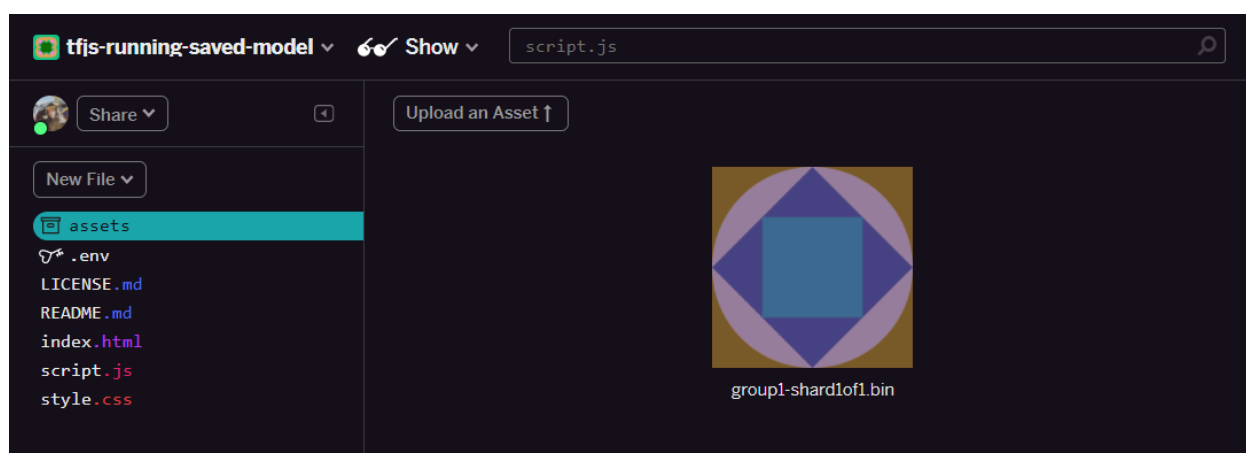
6. 在瀏覽器中使用轉換後的模型

代管轉換後的檔案

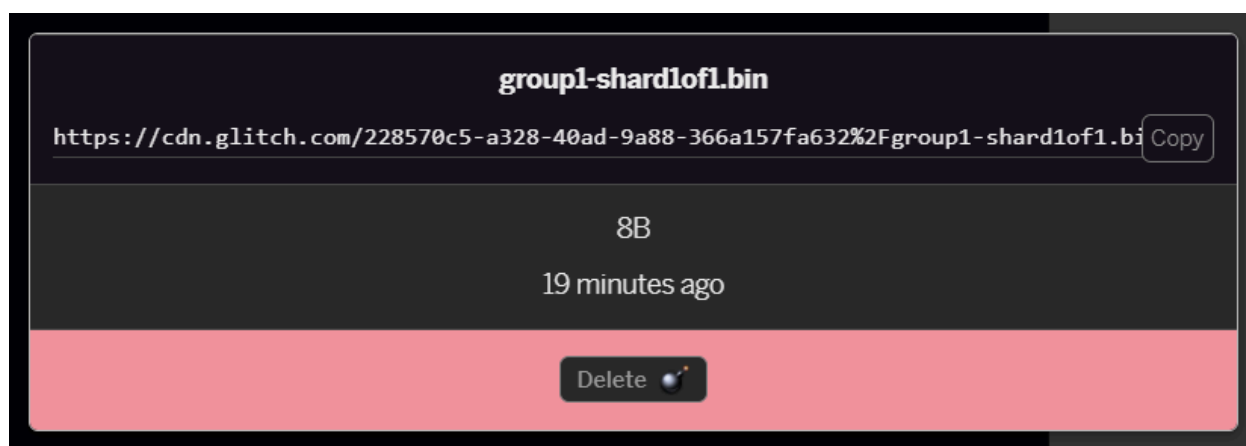
首先，我們必須將產生的 `model.json` 和 `*.bin` 檔案放在網路伺服器上，才能透過網頁存取這些檔案。在本示範中，我們將使用 [Glitch.com](https://glitch.com)，方便您跟著操作。不過，如果您有網路工程背景，也可以選擇在目前的 Ubuntu 伺服器執行個體上啟動簡易的 HTTP 伺服器，藉此完成這項作業。一切都由您決定！

將檔案上傳到 Glitch

1. 登入 [Glitch.com](https://glitch.com)
2. 使用這個連結複製我們的樣板 [TensorFlow.js 專案](#)。其中包含 HTML、CSS 和 JS 檔案的架構，這些檔案會匯入 TensorFlow.js 程式庫，供我們使用。
3. 按一下左側面板中的「素材資源」資料夾。
4. 按一下「上傳資產」，然後選取要上傳至這個資料夾的 `group1-shard1of1.bin`。上傳後應如下所示：



5. 按一下剛上傳的 `group1-shard1of1.bin` 檔案，即可複製該檔案位置的網址。現在請複製這個路徑，如下所示：



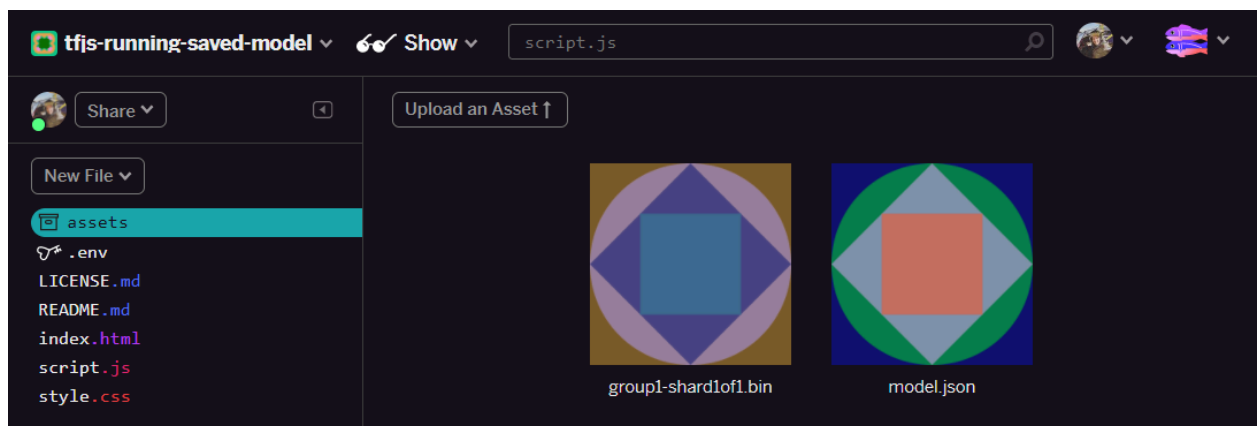
6. 現在請在本機使用偏好的文字編輯器編輯 `model.json`，然後搜尋 (使用 Ctrl+F) `group1-shard1of1.bin` 檔案，該檔案會出現在 `model.json` 檔案中。

將這個檔案名稱替換為您在步驟 5 複製的網址，但請刪除 Glitch 從複製路徑產生的開頭 <https://cdn.glitch.com/>。

編輯後，檔案應如下所示 (請注意，開頭的伺服器路徑已移除，因此只會保留上傳的檔案名稱)：

```
{
  "format": "layers-model",
  "generatedBy": "keras v2.4.0",
  "convertedBy": "TensorFlow.js Converter v2.6.0",
  "modelTopology": {
    "keras_version": "2.4.0",
    "backend": "tensorflow",
    "model_config": {
      "class_name": "Sequential",
      "config": {
        "name": "sequential",
        "layers": [
          {
            "class_name": "InputLayer",
            "config": {
              "batch_input_shape": [null, 1],
              "dtype": "float32",
              "sparse": false,
              "ragged": false,
              "name": "dense_input"
            }
          },
          {
            "class_name": "Dense",
            "config": {
              "name": "dense",
              "trainable": true,
              "batch_input_shape": [null, 1],
              "dtype": "float32",
              "units": 1,
              "activation": "linear",
              "use_bias": true,
              "kernel_initializer": {
                "class_name": "GlorotUniform",
                "config": {
                  "seed": null
                }
              },
              "bias_initializer": {
                "class_name": "Zeros",
                "config": {}
              },
              "kernel_regularizer": null,
              "bias_regularizer": null,
              "activity_regularizer": null,
              "kernel_constraint": null,
              "bias_constraint": null
            }
          }
        ]
      }
    },
    "training_config": {
      "loss": "mean_absolute_error",
      "metrics": [],
      "loss_weights": null,
      "sample_weight_mode": null,
      "weighted_metrics": null,
      "optimizer_config": {
        "class_name": "SGD",
        "config": {
          "name": "SGD",
          "learning_rate": 0.009999999776482582,
          "decay": 0.0,
          "momentum": 0.0,
          "nesterov": false
        }
      },
      "weightsManifest": [
        {
          "paths": [
            "228570c5-a328-40ad-9a88-366a157fa632%2Fgroup1-shard1of1.bin"
          ],
          "weights": [
            {
              "name": "dense/kernel",
              "shape": [1, 1],
              "dtype": "float32"
            },
            {
              "name": "dense/bias",
              "shape": [1],
              "dtype": "float32"
            }
          ]
        }
      ]
    }
  }
}
```

7. 現在請儲存並上傳這個編輯過的 `model.json` 檔案至 Glitch，方法是點選素材資源，然後按一下「上傳素材資源」按鈕 (重要)。如果您未使用實體按鈕，而是採用拖曳方式，系統會將檔案上傳為可編輯的檔案，而非上傳至 CDN，因此檔案不會位於相同資料夾中，TensorFlow.js 嘗試下載特定模型的二進位檔時，會假設相對路徑。如果操作正確，您應該會在 `assets` 資料夾中看到 2 個檔案，如下所示：



太好了！現在我們已準備好在瀏覽器中使用儲存的檔案和實際程式碼。

警告：如果您決定不使用 Glitch.com，而是自行執行網路伺服器，只要從網路伺服器上的相同目錄提供檔案，就不需要編輯 `model.json` 檔案。只要確保這些檔案位於相同資料夾，且可從用戶端程式碼存取即可。

載入模型

現在我們已代管轉換後的檔案，可以編寫簡單的網頁來載入這些檔案，並用於進行預測。在 Glitch 專案資料夾中開啟 `script.js`，並將這個檔案的內容替換為下列程式碼。請先變更 `const MODEL_URL`，指向您在 Glitch 上傳的 `model.json` 檔案所產生的 Glitch.com 連結：

script.js:

```
// Grab a reference to our status text element on the web page.
// Initially we print out the loaded version of TFJS.
const status = document.getElementById('status');
status.innerText = 'Loaded TensorFlow.js - version: ' + tf.version.tfjs;

// Specify location of our Model.json file we uploaded to the Glitch.com CDN.
const MODEL_URL = 'YOUR MODEL.JSON URL HERE! CHANGE THIS!';
// Specify a test value we wish to use in our prediction.
// Here we use 950, so we expect the result to be close to 950,000.
const TEST_VALUE = 950.0

// Create an asynchronous function.
async function run() {
  // Load the model from the CDN.
  const model = await tf.loadLayersModel(MODEL_URL);

  // Print out the architecture of the loaded model.
  // This is useful to see that it matches what we built in Python.
  console.log(model.summary());

  // Create a 1 dimensional tensor with our test value.
  const input = tf.tensor1d([TEST_VALUE]);

  // Actually make the prediction.
  const result = model.predict(input);

  // Grab the result of prediction using dataSync method
  // which ensures we do this synchronously.
  status.innerText = 'Input of ' + TEST_VALUE +
    'sqft predicted as $' + result.dataSync()[0];
}

// Call our function to start the prediction!
run();
```

將 `MODEL_URL` 常數變更為指向 `model.json` 路徑後，執行上述程式碼會產生如下所示的輸出內容。

請注意：如要執行程式碼，只要按一下 Glitch 網站頂端的「顯示」，然後選擇「新視窗」或「程式碼旁」，即可查看程式碼執行的即時預覽畫面。

TensorFlow.js Hello World

Input of 950sqft predicted as \$946602.375

如果檢查網頁瀏覽器的控制台 (按下 F12 鍵即可在瀏覽器中開啟開發人員工具)，我們也會看到已載入模型的模型說明，其中會列印：

Layer (type)	Output shape	Param #
=====		
dense_input (InputLayer)	[null,1]	0
=====		
dense (Dense)	[null,1]	2
=====		
Total params: 2		
Trainable params: 2		
Non-trainable params: 0		
=====		

與本程式碼研究室開頭的 Python 程式碼比較後，我們可以確認這是我們建立的相同網路，其中包含 1 個密集輸入和 1 個節點的稠密層。

恭喜！您剛才在網頁瀏覽器中執行轉換後的 Python 訓練模型！

7. 無法轉換的模型

有時，系統不支援轉換較複雜的模型，因為這些模型會編譯成較不常見的作業。TensorFlow.js 的瀏覽器版本是 TensorFlow 的完整重寫版本，因此我們目前不支援 TensorFlow C++ API 的所有低階運算 (有數千個)，但隨著我們成長，以及核心運算變得更加穩定，我們將逐步新增更多運算。

撰寫本文時，TensorFlow Python 中有一個函式 (`linalg.diag`) 會在匯出為 SavedModel 時產生不支援的運算元。如果我們嘗試在 Python 中轉換使用這項功能的 SavedModel (Python 支援產生的結果運算)，會看到類似下方的錯誤訊息：

```
jasonmayes@tfjs-converter-codelab-test: ~/tmp/saved_model
2020-11-13 21:47:50.485080: I tensorflow/stream_executor/cuda/cuda_diagnostics.cc:156] kernel driver does not appear to be running on this host (tfjs-
converter-codelab-test): /proc/driver/nvidia/version does not exist
2020-11-13 21:47:50.485416: I tensorflow/core/platform/cpu_feature_guard.cc:142] This TensorFlow binary is optimized with oneAPI Deep Neural Network L
ibrary (oneDNN) to use the following CPU instructions in performance-critical operations: AVX2 AVX512F FMA
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
2020-11-13 21:47:50.493314: I tensorflow/core/platform/profile_utils/cpu_utils.cc:104] CPU Frequency: 2000000000 Hz
2020-11-13 21:47:50.493845: I tensorflow/compiler/xla/service/service.cc:168] XLA service 0x5146cb0 initialized for platform Host (this does not guar
antee that XLA will be used). Devices:
2020-11-13 21:47:50.493899: I tensorflow/compiler/xla/service/service.cc:176] StreamExecutor device (0): Host, Default Version
2020-11-13 21:47:50.526275: I tensorflow/core/grappler/devices.cc:69] Number of eligible GPUs (core count >= 8, compute capability >= 0.0): 0
2020-11-13 21:47:50.526425: I tensorflow/core/grappler/clusters/single_machine.cc:356] Starting new session
2020-11-13 21:47:50.532118: I tensorflow/core/grappler/optimizers/meta_optimizer.cc:816] Optimization results for grappler item: graph to optimize
2020-11-13 21:47:50.532169: I tensorflow/core/grappler/optimizers/meta_optimizer.cc:818] function_optimizer: Graph size after: 15 nodes (11), 14 edg
es (11), time = 0.713ms.
2020-11-13 21:47:50.532183: I tensorflow/core/grappler/optimizers/meta_optimizer.cc:818] function_optimizer: function_optimizer did nothing. time =
0.014ms.
Traceback (most recent call last):
  File "/home/jasonmayes/.local/bin/tensorflowjs_converter", line 8, in <module>
    sys.exit(pip_main())
  File "/home/jasonmayes/.local/lib/python3.6/site-packages/tensorflowjs_converters/converter.py", line 757, in pip_main
    main([' '.join(sys.argv[1:])])
  File "/home/jasonmayes/.local/lib/python3.6/site-packages/tensorflowjs_converters/converter.py", line 761, in main
    convert(argv[0].split(' '))
  File "/home/jasonmayes/.local/lib/python3.6/site-packages/tensorflowjs_converters/converter.py", line 699, in convert
    experiments=args.experiments)
  File "/home/jasonmayes/.local/lib/python3.6/site-packages/tensorflowjs_converters/tf_saved_model_conversion_v2.py", line 614, in convert_tf_saved_mo
del
    initializer_graph=frozen_initializer_graph)
  File "/home/jasonmayes/.local/lib/python3.6/site-packages/tensorflowjs_converters/tf_saved_model_conversion_v2.py", line 146, in optimize_graph
    ', '.join unsupported))
ValueError: Unsupported Ops in the model before optimization
MatrixDiagV3
jasonmayes@tfjs-converter-codelab-test:~/tmp/saved_model$
```

如紅色醒目顯示部分所示，`linalg.diag` 呼叫編譯後會產生名為 `MatrixDiagV3` 的運算，但撰寫本程式碼研究室時，TensorFlow.js 網路瀏覽器並不支援這個運算。

請注意：如要查看 [目前支援的 TensorFlow.js 運算清單](#)，請按這裡。請注意，這僅適用於 TensorFlow.js 的用戶端網路瀏覽器實作，不適用於 TensorFlow.js 的 Node.js 版本 (該版本使用與 Python 相同的 C++ 繫結)，因此在伺服器端使用 TensorFlow.js 時，所有儲存的模型都可以在 Node.js 中執行，無須轉換。

建議採取的行動

具體有兩種方法。

1. 在 TensorFlow.js 中實作這個缺少的運算子 - 我們是開放原始碼專案，歡迎貢獻新運算子等內容。請參閱這份指南，瞭解如何為 TensorFlow.js 編寫新的運算。如果成功完成這項操作，您就可以在指令列轉換器上使用 `skip_op_check` 旗標，忽略這項錯誤並繼續轉換 (系統會假設您建立的新版 TensorFlow.js 版本支援缺少的運算)。
2. 判斷您匯出的 `savedmodel` 檔案中，哪個部分的 Python 程式碼產生不支援的作業。在少量程式碼中，這可能很容易找到，但在更複雜的模型中，這可能需要相當多的調查，因為目前沒有方法可以識別產生特定運算的高階 Python 函式呼叫 (一旦採用 `savedmodel` 檔案格式)。不過，找到後您或許可以改用其他支援的方法。

步驟 8 · 約 0 分鐘

8. 恭喜

恭喜！您已完成第一步，在網路瀏覽器中透過 TensorFlow.js 使用 Python 模型！

重點回顧

在本程式碼研究室中，我們學到如何：

1. 設定 Linux 環境，安裝以 Python 為基礎的 TensorFlow
2. 匯出 Python 「SavedModel」
3. 安裝 TensorFlow.js 指令列轉換器
4. 使用 TensorFlow.js 指令列轉換器建立必要的用戶端檔案
5. 在實際的網頁應用程式中使用產生的檔案
6. 找出無法轉換的模型，以及日後轉換模型時需要導入的項目。

後續步驟

別忘了在所有使用 [#MadeWithTFJS](#) 建立的內容中標記我們，就有機會在社群媒體上曝光，甚至在日後的 TensorFlow 活動中展示。我們很期待您在瀏覽器中轉換及使用用戶端！

更多 TensorFlow.js 程式碼研究室，深入瞭解相關主題

- [使用 TensorFlow.js 從頭開始編寫類神經網路](#)
- [建立可偵測物體的智慧型網路攝影機](#)
- [使用 TensorFlow.js 中的遷移學習功能進行自訂圖片分類](#)

值得一訪的網站

- [TensorFlow.js 官方網站](#)
 - [TensorFlow.js 預先建構模型](#)
 - [TensorFlow.js API](#)
 - [社群的#MadeWithTFJS 範例](#)
-